MediSlot — Capstone Project Brief

Doctor Appointment Booking & Management Platform — *AWS-Native Cloud Application*

Start Date	Monday, 12 October 2026
Completion Date	Sunday, 29 November 2026
Duration	7 weeks (35 working days)
Milestones	7 checkpoints
Cloud Platform	AWS
Language	Python
Infrastructure as Code	AWS CloudFormation
CI/CD	GitHub Actions
Version Control	GitHub

Objective and Business Context

Healthcare appointment booking is one of the most common real-world backend problems, and one of the most unforgiving. Two patients tapping "Book" on the last available slot at the same instant is not an edge case — it is routine traffic. A system that double-books a doctor has failed in a way that is immediately visible to real people.

The previous capstone covered the breadth of AWS: many services, each used in a straightforward way. This project covers depth. It takes a domain that looks like simple CRUD and exposes the concurrency, consistency, and time-handling problems that live underneath it.

The defining characteristic of this project is that **correctness is objectively testable**. A reviewer can fire twenty simultaneous booking requests at a single slot and verify whether the system held the line. There is no room for a system that works when one person clicks slowly. That gap — between "it works when I try it" and "it is correct under load" — is what this project is built to close.

Participants will also build their first complete multi-role product: two distinct user types, two separate dashboards, authentication, role-based authorization, search, financial rules around refunds, and a notification system. By the end they will have shipped something that resembles a real product rather than an exercise.

Problem Statement / Use Case

MediSlot is the backend and operations platform for an online doctor appointment service. It serves two distinct user groups through separate interfaces.

Doctors need to

- Register and log in to the platform
- Maintain a professional profile: name, licence number, specialisation, years of experience, qualifications, hospital or clinic name and address, languages spoken

- Define consultation types with duration and fee
- Publish recurring weekly availability, and declare blackout dates for leave or conference travel
- See their upcoming schedule, who is booked into each slot, and their day view
- Cancel their own slots when necessary, with patients automatically refunded in full
- Mark appointments as completed or no-show

Patients need to

- Register and log in to the platform
- Maintain a basic profile: name, date of birth, gender, contact details, city, emergency contact
- Search for doctors by name, hospital name, specialisation, or city
- View a doctor's available slots and consultation fees
- Book an appointment
- View their upcoming and past appointments
- Reschedule an appointment to a different slot
- Cancel an appointment, with the refund amount determined by how far in advance they cancel
- Join a waitlist when a doctor is fully booked, and be notified automatically if a slot opens up

The system must guarantee

- A slot is **never** booked beyond its capacity, regardless of how many patients attempt it simultaneously
- A retried booking request never produces two appointments
- A reschedule leaves the patient holding exactly one appointment — never zero, never two
- A freed slot is offered to exactly one waitlisted patient at a time
- Refund amounts are calculated correctly and consistently against the cancellation window
- Every appointment change — booked, updated, cancelled — produces a notification
- Patients can only see and modify their own appointments; doctors only their own schedule

---

Frozen Rule Set

These rules are fixed. They are deliberately simplified and **not open to reinterpretation** — effort belongs in the engineering, not in product design. Anything not stated here is out of scope. Every threshold, percentage, and duration below must be stored in **SSM Parameter Store**, never hardcoded.

Authentication and Token Rules

Authentication is implemented from scratch. No managed identity service is permitted.

- Passwords are hashed with **bcrypt** at a work factor of 12 or higher. Storing plaintext, reversibly encrypted, or unsalted-hash passwords is an automatic failure
- Password policy enforced at registration: minimum 10 characters, at least one letter and one digit
- Email addresses are unique across each role. A doctor and a patient may not share an email address
- **Access tokens** are JWTs signed with HS256, expiring in **15 minutes**
- Access token payload carries exactly: `sub` (userId), `role` (`doctor` or `patient`), `iat`, `exp`, and `jti`
- **Refresh tokens** are opaque random strings, expiring in **7 days**, stored server-side so they can be revoked

- The JWT signing secret lives in **SSM Parameter Store as a SecureString** and is read at Lambda cold start, never hardcoded
- The Lambda Authorizer verifies signature, expiry, and issuer on every request, then returns an IAM policy plus `userId` and `role` in the authorizer context
- Downstream handlers read the acting user **only** from the authorizer context. Reading `userId` or `role` from a request body, query string, or header is an automatic failure
- Logout deletes the refresh token and adds the access token's `jti` to a denylist with a TTL matching the token's remaining lifetime
- A token presented after logout must be rejected
- An expired, tampered, or wrongly-signed token must return `401`. A valid token for the wrong role must return `403`
- Token lifetimes and the bcrypt work factor are SSM-configurable

Consultation Types

A doctor defines one or more consultation types, each with a fixed duration and fee.

Consultation Type	Duration	Slot Capacity
Quick Consult	15 min	1
Standard Consult	30 min	1
Detailed Consult	60 min	1
Health Camp Session	60 min	2 to 10

Health Camp Sessions exist specifically so that concurrency must handle a capacity greater than one. Exactly `capacity` bookings succeed; the rest are rejected.

Availability and Slot Rules

- A doctor has a **home timezone** recorded as an IANA identifier, for example `Asia/Kolkata`
- Availability is defined as **recurring weekly patterns**: day of week, start time, end time, consultation type — all expressed in the doctor's local time
- A doctor may declare **blackout dates** that suppress all slots on those dates
- Slots are materialised into a **rolling 30-day booking window**
- Slot start times align to the consultation duration from the pattern start time
- A daily scheduled job rolls the window forward by one day
- Materialisation must be **idempotent** — re-running it for an already-processed date produces no duplicate slots
- Slot times are stored in **UTC**, with the originating timezone recorded alongside
- Patterns must materialise correctly across a **daylight saving transition**: 09:00–17:00 local means 09:00–17:00 local on every date, even when the UTC offset shifts

Booking Rules

- A patient books one seat in a slot
- A slot accepts bookings until `booked_count` reaches `capacity`. Further attempts are rejected with `409 Conflict`

- Under concurrent load, **exactly** `capacity` **bookings must succeed**. Overbooking by even one seat is a correctness failure
- A patient may hold at most **one appointment per slot**
- A patient may not book a slot whose start time is in the past
- A patient may hold at most **3 upcoming appointments** with the same doctor
- Every booking request carries a client-supplied `Idempotency-Key` header. Replaying a key returns the original appointment without creating a second
- Appointment states: `CONFIRMED` → `COMPLETED` | `CANCELLED_BY_PATIENT` | `CANCELLED_BY_DOCTOR` | `NO_SHOW`

Cancellation and Refund Rules

Refund is calculated from the time of cancellation relative to the appointment start.

Cancellation Timing	Refund	Deduction
More than 48 hours before	100%	0%
Between 24 and 48 hours before	75%	25%
Between 6 and 24 hours before	50%	50%
Less than 6 hours before	0%	100%
Cancelled by the doctor	**100%**	**0%** — regardless of timing
Patient marked `NO_SHOW`	0%	100%

- Refund amounts are **recorded in a refund ledger**, not actually charged. No payment gateway is involved
- Cancellation frees the seat and triggers waitlist promotion
- All window boundaries and percentages are SSM-configurable

Reschedule Rules

- A reschedule moves an appointment from its current slot to a target slot
- The operation must be **atomic from the patient's perspective**: either they hold the target slot, or they still hold the original. Never neither, never both
- If the target slot is full, the reschedule fails and the original appointment remains `CONFIRMED`
- An appointment may be rescheduled at most **2 times**; further attempts are rejected
- Rescheduling within 6 hours of the appointment start is not permitted
- A reschedule carries no refund or fee implication

Waitlist Rules

- When a slot is full, a patient may join its **waitlist**, ordered by join time
- When a seat frees, the system issues a **hold offer** to the first waitlisted patient
- A hold offer reserves the seat exclusively for **15 minutes**. During the hold the seat is not bookable by anyone else
- If accepted, an appointment is created and the patient leaves the waitlist
- If the offer expires unaccepted, it passes to the next patient in line
- The same freed seat must **never** be offered to two patients simultaneously

- A patient may be on at most **5 waitlists** at once
- Hold duration is SSM-configurable

Notification Rules

A notification is dispatched on each of the following events:

Event	Recipient
Appointment booked	Patient and doctor
Appointment rescheduled	Patient and doctor
Appointment cancelled by patient	Patient and doctor
Appointment cancelled by doctor	Patient
Waitlist hold offer issued	Patient
Waitlist hold offer expired	Patient
Reminder, 24 hours before	Patient
Reminder, 2 hours before	Patient

- Each notification is sent **exactly once** per event, even if the dispatcher runs repeatedly
- Notifications are published to SNS and verified there. Real email or SMS delivery is out of scope
- An appointment still `CONFIRMED` **30 minutes after** its slot end is automatically marked `NO_SHOW`

Feature to AWS Service Mapping

This is the recommended service for each feature. Participants may deviate, but a deviation must be justified in the design review.

Identity and Access

Authentication is **built, not bought**. No managed identity service is used. Participants implement registration, login, token issuance, and token validation themselves as Lambda functions. This is deliberate — the objective is to understand how token-based authentication actually works rather than to configure a service that hides it.

Feature	Recommended Services	Why
Doctor and patient registration	**API Gateway + Lambda + RDS MySQL**	A custom `register` Lambda hashes the password with bcrypt and writes the user record. Plaintext or reversibly encrypted passwords are an automatic failure
Login and token issuance	**API Gateway + Lambda + SSM Parameter Store**	A custom `login` Lambda verifies the password hash and signs a JWT using a secret held as an SSM SecureString
Token validation	**API Gateway + Lambda Authorizer**	A custom `authorizer` Lambda verifies the JWT signature and expiry, then resolves `userId` and `role` into the authorizer context
Refresh tokens	**Lambda + DynamoDB with TTL**	Refresh tokens stored server-side so they can be revoked. Access tokens stay short-lived

Feature	Recommended Services	Why
Role separation (doctor vs patient)	**JWT custom claim**	`role` carried as a signed claim. Never read from a request body or query parameter
Logout and revocation	**Lambda + DynamoDB**	Refresh token deleted, and the access token's `jti` added to a short-lived denylist
Signing secret storage	**SSM Parameter Store SecureString**	Rotatable without a code change. Never in source, never in an environment variable literal

Profiles and Catalogue

Feature	Recommended Services	Why
Doctor profile CRUD	**API Gateway + Lambda + RDS MySQL**	Highly relational — doctors link to hospitals, specialisations, consultation types. Needs joins and referential integrity
Patient profile CRUD	**API Gateway + Lambda + RDS MySQL**	Same reasoning. Also the system of record for anything auditable
Consultation type definitions	**API Gateway + Lambda + RDS MySQL**	Foreign-keyed to the doctor, queried alongside profile
Profile photo upload	**S3 + pre-signed URLs from Lambda**	Binary content never passes through API Gateway
Doctor search by name, hospital, specialisation, city	**API Gateway + Lambda + RDS MySQL with indexes**	Multi-attribute filtered search is a relational strength and a DynamoDB weakness

Availability and Slots

Feature	Recommended Services	Why
Weekly availability patterns	**API Gateway + Lambda + RDS MySQL**	Low volume, relational, rarely changes
Blackout dates	**API Gateway + Lambda + RDS MySQL**	Same
Slot materialisation job	**EventBridge Scheduler + Lambda**	Scheduled daily, rolls the 30-day window forward
Materialised slot storage	**DynamoDB**	High read volume, point lookups by slot, and the contended write target. Conditional writes are the core requirement
Slot availability lookup	**API Gateway + Lambda + DynamoDB**	Single-digit millisecond reads, queried by doctor and date range via GSI

Booking — the core of the project

Feature	Recommended Services	Why
Concurrency-safe booking	**DynamoDB conditional writes with a version attribute**	Optimistic concurrency control. The only mechanism here that gives a hard guarantee against overbooking without serialising traffic

Feature	Recommended Services	Why
Appointment system of record	**RDS MySQL**	Permanent, auditable, joinable against patients, doctors, and refunds
Idempotency	**DynamoDB with TTL**	Keyed on the client-supplied `Idempotency-Key`, stores the original response, expires after 24 hours
Reschedule across two slots	**Step Functions + Lambda**	A saga with explicit compensation. Claim target, release original, compensate on partial failure
Refund calculation	**Lambda + RDS MySQL + SSM Parameter Store**	Business rules from parameters, amounts written to a refund ledger

Waitlist

Feature	Recommended Services	Why
Waitlist queue per slot	**DynamoDB**	Ordered by join time using a sort key; cheap appends and reads
Hold offers with expiry	**DynamoDB with TTL + EventBridge Scheduler sweeper**	TTL alone cannot drive a state transition, so a sweeper handles pass-on logic
Seat-freed trigger	**EventBridge**	`SlotSeatFreed` published on cancellation, consumed independently by the waitlist handler
Offer notification	**SQS + Lambda + SNS**	Buffered so a notification failure never blocks the booking path

Notifications

Feature	Recommended Services	Why
Domain event publication	**EventBridge**	One event, many independent consumers. Adding a consumer requires no change to the producer
Consumer buffering	**SQS with DLQ per consumer**	Isolates failure. A broken notification handler does not retry-storm the booking API
Notification delivery	**SNS**	Email topic per recipient type; alarm delivery reuses the same service
Appointment reminders	**EventBridge Scheduler + Lambda + SNS**	Scheduled sweep finds due reminders and dispatches exactly once
Exactly-once dispatch	**DynamoDB dispatch log**	Conditional write on `appointmentId#interval` prevents duplicate sends

Dashboards

Feature	Recommended Services	Why
Single web application	**EC2 `t2.micro` + Python Flask**	One instance, one Flask app. Landing page offers "Login as Doctor" or "Login as Patient", then routes to the chosen portal

Feature	Recommended Services	Why
Role-based routing and views	**Flask blueprints + server-side sessions**	Two route namespaces, `/doctor/*` and `/patient/*`, guarded by the role claim from the JWT held in the session
Static assets	**S3**	CSS, JS, images served from the bucket
Instance bootstrap	**EC2 UserData + S3**	Application pulled from S3 on boot so the instance is disposable

Platform and Operations

Feature	Recommended Services	Why
Network isolation	**VPC, public and private subnets, security groups**	RDS in private subnet with no internet route
Infrastructure definition	**CloudFormation**	Parameterised stacks with cross-stack references
Deployment automation	**GitHub Actions + S3**	Lambda packages to S3, stacks updated via Change Sets
Permissions	**IAM**	Per-function execution roles, OIDC trust for GitHub Actions
Logs, metrics, dashboards, alarms	**CloudWatch**	Structured logs, custom business metrics, two dashboards, alarms to SNS
Request tracing	**X-Ray**	End-to-end trace of a booking and a reschedule, including compensation
Configuration and secrets	**SSM Parameter Store**	All thresholds, windows, percentages, credentials
Cost control	**AWS Budgets**	Alert threshold, cost allocation tags on every resource

---

Scope and Expected Deliverables

In Scope

Authentication and Authorization

- Custom `register` Lambda per role, hashing passwords with bcrypt and enforcing the password policy
- Custom `login` Lambda verifying credentials and issuing a signed JWT access token plus an opaque refresh token
- Custom `refresh` and `logout` Lambdas for token rotation and revocation
- Lambda Authorizer on every protected endpoint, verifying the JWT signature, expiry, and role claim
- Role enforcement: doctor-only endpoints reject patient tokens and vice versa
- The acting user is always derived from the token, **never** from a request body field
- Authorizer cache TTL configured and justified

Doctor APIs

- `POST /auth/doctors/register`, `POST /auth/doctors/login`
- `POST /auth/refresh`, `POST /auth/logout` — shared by both roles

- `GET /doctors/me`, `PUT /doctors/me` — profile management
- `POST /doctors/me/photo` — pre-signed upload URL
- `POST /doctors/me/consultation-types`, `GET`, `PUT`, `DELETE`
- `PUT /doctors/me/availability` — weekly patterns
- `POST /doctors/me/blackouts`, `GET`, `DELETE`
- `GET /doctors/me/schedule?date=` — day view with booked patients
- `GET /doctors/me/appointments?from=&to=&status=`
- `DELETE /doctors/me/slots/{slotId}` — cancel a slot, full refund to all affected patients
- `POST /doctors/me/appointments/{id}/outcome` — mark `COMPLETED` or `NO_SHOW`

Patient APIs

- `POST /auth/patients/register`, `POST /auth/patients/login`
- `GET /patients/me`, `PUT /patients/me`, `DELETE /patients/me`
- `GET /doctors?name=&hospital=&specialisation=&city=` — search with pagination
- `GET /doctors/{doctorId}` — public profile with consultation types and fees
- `GET /doctors/{doctorId}/slots?from=&to=&consultationTypeId=` — availability
- `POST /appointments` — book, `Idempotency-Key` header required
- `GET /appointments` — the caller's own appointments, upcoming and past
- `GET /appointments/{id}`
- `PATCH /appointments/{id}` — reschedule to a target slot
- `DELETE /appointments/{id}` — cancel, returns the computed refund amount
- `POST /slots/{slotId}/waitlist`, `DELETE /slots/{slotId}/waitlist`
- `GET /patients/me/waitlists`
- `POST /waitlist-offers/{offerId}/accept`
- `GET /patients/me/refunds` — refund ledger

Booking Engine

- Optimistic concurrency control using DynamoDB conditional writes and a version attribute
- Bounded retry with backoff on conflict, with retries counted as a metric
- Capacity enforced exactly, including for multi-capacity Health Camp Sessions
- Idempotency store returning the original response on key replay
- Appointment written to RDS as the system of record
- `AppointmentBooked` published to EventBridge

Reschedule Saga

- Orchestrated by Step Functions with explicit compensation paths
- Claim target seat, release original seat, compensate if the release fails after a successful claim
- Reschedule attempt limit and 6-hour cutoff enforced
- `AppointmentRescheduled` published on success

Cancellation and Refunds

- Refund tier calculation against SSM-configured windows
- Doctor-initiated cancellation always refunds in full
- Refund written to a ledger in RDS

- `AppointmentCancelled` and `SlotSeatFreed` published

Waitlist and Hold Offers

- Waitlist join, leave, and listing
- Hold offer issued on `SlotSeatFreed` to the first waitlisted patient
- Held seats invisible to normal booking for the hold duration
- Offer acceptance converts to a confirmed appointment
- Expiry sweeper passes the seat to the next patient in line
- Exclusivity guaranteed — never two live offers for the same seat

Notification System

- EventBridge as the domain event bus
- SQS-buffered consumers with per-consumer DLQs
- SNS delivery for all eight notification events in the Frozen Rule Set
- Exactly-once dispatch enforced through a DynamoDB dispatch log
- Reminder dispatcher running on schedule

Scheduled Jobs — every one must be safe to run twice

- Slot materialiser — daily
- Hold offer expiry sweeper — every minute
- Reminder dispatcher — every five minutes
- No-show sweeper — every fifteen minutes

Single Web Application (EC2)

One Flask application on one `t2.micro` instance, serving both user types.

- **Landing page** at `/` presenting two choices: **Login as Doctor** or **Login as Patient**
- Selecting a role routes to the corresponding login form, `/doctor/login` or `/patient/login`
- On successful login the app calls the platform's own `login` API, receives the JWT, and stores it in a server-side session
- After login the user lands on the portal matching their role and sees only that portal's navigation
- **Doctor portal** (`/doctor/*`): profile editor, consultation type management, availability editor, blackout management, day and week schedule view, patient list per slot, appointment outcome marking, slot cancellation
- **Patient portal** (`/patient/*`): profile editor, doctor search with filters, slot browser, booking flow, upcoming and past appointments, active waitlists with pending offers, refund history
- Route guards enforce the role claim — a patient session reaching any `/doctor/*` route is redirected to the landing page, and the reverse also holds
- Registration links available from both login forms
- Logout clears the session and calls the platform's `logout` API to revoke the refresh token
- The web app is a **pure API client**. It holds no database credentials and makes no direct database connection. Every read and write goes through API Gateway

Observability

- Structured JSON logging with a correlation ID generated at the edge and threaded through every log line and event payload

- X-Ray tracing across API Gateway, Lambda, Step Functions, and downstream calls
- CloudWatch dashboard `medislot-operations` with widgets for:
  - API Gateway: request count, 4xx rate, 5xx rate, latency p50 and p99
  - Lambda: invocations, errors, duration p50 and p99, throttles per function
  - DynamoDB: consumed capacity, throttled requests, conditional check failures
  - SQS: messages visible per queue, DLQ depth, oldest message age
  - Step Functions: executions started, failed, compensation path entries
  - RDS: CPU, connections, free storage
  - Authentication: login successes, login failures, token validation rejections
- Custom business metrics under the `medislot` namespace: `AppointmentsBooked`, `BookingsRejectedFull`, `BookingConflictRetries`, `ReschedulesSucceeded`, `ReschedulesCompensated`, `CancellationsByTier`, `RefundAmountTotal`, `WaitlistPromotions`, `OffersExpired`, `SlotUtilisationPercent`, `NoShowRate`, `NotificationsDispatched`
- Alarms with SNS delivery for: DLQ depth above zero, any Step Functions compensation entry, conditional check failure rate above threshold, API 5xx rate, RDS CPU above 70%, any Lambda error rate above threshold
- Dashboard and all alarms defined in the monitoring CloudFormation stack, never created by hand

Infrastructure as Code

- Custom VPC with public and private subnets
- RDS in a private subnet with no internet route
- Infrastructure split into separate parameterised stacks with cross-stack references
- Lambda packages built and uploaded to S3, referenced from templates
- All configuration in SSM Parameter Store
- No customer-managed KMS keys; AWS-managed keys only

Source Control Workflow

- `master` protected: no direct commits, pull request required, at least one peer approval, CI passing before merge
- All work on short-lived feature branches following the naming convention
- Branch protection rules configured in Milestone 1 and verified at the Milestone 2 checkpoint

CI/CD Pipeline

- GitHub Actions on pull request: lint → unit tests → integration tests, as a required status check
- GitHub Actions on merge to `master`: lint → unit tests → package → upload → deploy via Change Sets → integration tests → smoke test
- Stack updates exclusively via CloudFormation Change Sets
- OIDC-based AWS authentication preferred over long-lived access keys
- Secret scanning and IaC scanning in the pipeline, blocking on findings

Automated Testing

- Unit tests for refund tier calculation, slot materialisation across DST, reschedule limit enforcement, waitlist ordering, and capacity arithmetic
- Integration tests executing against deployed infrastructure
- Four **mandatory named tests**, each gating the pipeline:

1. **Concurrency test** — 20+ simultaneous bookings at a capacity-3 Health Camp slot; exactly 3 succeed
2. **Idempotency test** — same `Idempotency-Key` replayed; exactly one appointment created
3. **Reschedule saga test** — forced failure during release; patient holds exactly one appointment
4. **Waitlist exclusivity test** — one freed seat, three waitlisted patients; only one live offer at any moment

Disaster Recovery

- Automated RDS backups with a documented retention period
- DynamoDB point-in-time recovery enabled
- A **restore drill** performed and documented with a measured restore time

Stretch Scope — for distinction, not required

- Second environment (`prod`) deployed from the same templates with different parameters
- API versioning with a breaking change shipped while the previous version remains functional
- A live schema migration with backfill applied to a running environment

Out of Scope

- Payment gateway integration — fees and refunds are recorded, never charged
- Real email or SMS delivery — notifications publish to SNS and are verified there
- Video consultation, chat, or prescription management
- Medical records beyond a free-text notes field on the patient profile
- Mobile applications
- Multi-region or multi-AZ deployment
- Recurring or series appointments
- Any rule not listed in the Frozen Rule Set

Expected Deliverables

#	Deliverable	Description
1	**GitHub Repository**	CloudFormation templates, Lambda source, GitHub Actions workflows, both dashboards, test suites — commit history distributed across all seven weeks
2	**Architecture Diagram**	All services, data flows, VPC layout, security boundaries, event topology, CI/CD pipeline
3	**Data Model Document**	RDS ERD with tables, indexes, constraints. DynamoDB key design per table with the access patterns each serves, and a written defence of the relational-versus-key-value split
4	**Concurrency Design Note**	How overbooking is prevented, how the reschedule saga guarantees exactly-one-appointment, how waitlist exclusivity is enforced, and what guarantees the system does and does not provide
5	**Timezone Design Note**	How availability is stored and materialised, with a worked example crossing a DST boundary
6	**API Documentation**	Every endpoint with method, path, auth requirement, request and response shapes, and error codes
7	**Test Report**	Results of the four mandatory tests, including the observed conflict-retry distribution under load

#	Deliverable	Description
8	**Deployment Runbook**	Zero-to-running instructions for a fresh account: JWT signing secret provisioning, GitHub secrets, stack order, web app deployment, full teardown
9	**DR Drill Record**	Documented restore exercise with elapsed time and verification steps
10	**Cost Review**	Resource-level cost breakdown with observations
11	**Walkthrough Demo**	30-minute live demonstration covering both dashboards and all four correctness guarantees

---

Technologies and Concepts to Be Applied

AWS Services

Service	Role in MediSlot
VPC	Public subnet for EC2 dashboards, private subnet for RDS; security groups as per-resource firewalls
IAM	Per-function execution roles, least-privilege policies, OIDC trust relationship for GitHub Actions
Custom auth Lambdas	Registration, login, token refresh, logout, and a JWT-validating Lambda Authorizer — all hand-built, no managed identity service
CloudFormation	Entire infrastructure as code — parameterised stacks, cross-stack references, Change Set updates
API Gateway	REST API, request body validation, Lambda Authorizer, per-method throttling
Lambda	All compute — authorizer, profile handlers, search, slot materialiser, booking engine, cancellation and refund, waitlist manager, notification dispatchers, scheduled sweepers
RDS MySQL	System of record — doctors, patients, hospitals, specialisations, consultation types, availability patterns, blackouts, appointments, refund ledger
DynamoDB	Materialised slots with conditional writes, waitlist queues, hold offers with TTL, idempotency store, notification dispatch log
SQS	Per-consumer buffering with dedicated DLQs
SNS	All patient and doctor notifications, plus CloudWatch alarm delivery
EventBridge	Domain event bus for appointment and slot lifecycle events
EventBridge Scheduler	Slot materialiser, offer expiry sweeper, reminder dispatcher, no-show sweeper
Step Functions	Reschedule saga with explicit compensation paths
S3	Lambda deployment packages, profile photos, dashboard static assets
EC2	`t2.micro` instance hosting both Flask dashboards
CloudWatch	Logs, custom metrics, operations dashboard, alarms
X-Ray	Distributed tracing across booking and reschedule paths
SSM Parameter Store	All configuration — refund windows and percentages, hold duration, booking window size, caps, credentials

Service	Role in MediSlot
AWS Budgets	Spend ceiling alerting with cost allocation tags
GitHub + GitHub Actions	Source control and CI/CD pipeline

Engineering Concepts

- **Optimistic concurrency control** — conditional writes, version attributes, bounded retry with backoff, conflict observability
- **Capacity-bounded allocation** — multi-capacity slots where exactly `capacity` writers succeed under arbitrary contention
- **Distributed transactions** — the saga pattern, compensating actions, orchestration versus choreography
- **Exclusive reservation** — hold offers that make a resource invisible to other claimants for a bounded window
- **Idempotency** — client-supplied keys, result caching, replay safety under network retry
- **Exactly-once effects** — dispatch logs preventing duplicate side effects from repeated job runs
- **Temporal correctness** — timezone-aware scheduling, DST boundary handling, UTC storage with local-time semantics
- **Event-driven architecture** — domain events, independently scaling consumers, buffering, DLQ-based failure capture
- **Time-driven state machines** — scheduled transitions, expiry sweeps, automatic state closure
- **Role-based authorization** — identity and role resolved from the token, never from request payload
- **Polyglot persistence** — choosing relational versus key-value per access pattern and defending the split
- **Relational data modelling** — normalisation, foreign keys, composite indexes for multi-attribute search
- **NoSQL data modelling** — access-pattern-first key design, GSI strategy, TTL for lifecycle management
- **Infrastructure as code** — stack decomposition, parameterisation, Change Set discipline, drift management
- **CI/CD** — automated pipeline, test gating, packaging, deployment ordering
- **Observability** — structured logging, correlation IDs, distributed tracing, business-level metrics, actionable alarms
- **Operational readiness** — runbooks, DR drills, cost accountability

---

Source Control and Collaboration Workflow

Apprentices implement individually, but the repository is run as a shared team repository. The branching and review discipline below is **mandatory and assessed**. Working alone is not a reason to skip code review — reviewing a peer's work and having your own reviewed is a core deliverable, not overhead.

Branching Policy

- `master` is the **protected default branch**. It always reflects a deployable state
- **No direct commits to `master`. Ever.** Not for a typo fix, not for a one-line change, not under deadline pressure
- All work happens on a **feature branch** cut from the latest `master`
- Feature branches are **short-lived**. A branch open for more than three days is a signal the work was scoped too large

Branch Naming

Type	Pattern	Example
Feature	`feature/<short-description>`	`feature/booking-concurrency-control`
Bug fix	`fix/<short-description>`	`fix/refund-tier-boundary`
Infrastructure	`infra/<short-description>`	`infra/monitoring-stack-alarms`
Documentation	`docs/<short-description>`	`docs/concurrency-design-note`

Commit Discipline

- Commits are made **on the feature branch**, never on `master`
- Commit messages describe **what changed and why**, not "wip", "fix", "update", or "changes"
- Commit as work progresses, not in a single dump at the end of the day. Commit history is reviewed as evidence of how the work actually unfolded
- Never commit credentials, `.env` files, local state, or build artefacts. A `.gitignore` is set up in Milestone 1

Pull Request Requirements

Every change reaches `master` through a pull request. A PR must:

- Target `master` from a feature branch
- Have a title stating what it does
- Have a description covering: what changed, why, how it was tested, and anything the reviewer should look at closely
- Pass the **full CI pipeline** — lint, unit tests, and integration tests all green. A red pipeline is not merged, regardless of urgency
- Carry **at least one approval from a peer apprentice** before merge
- Be merged by its author after approval, not by the reviewer

Peer Review Expectations

- Every apprentice is both an author and a reviewer. Review load is expected to be roughly balanced across the group
- A review means **reading the code and responding to it**. An approval with no comments on a 300-line change will be questioned
- Reviewers look for: correctness, hardcoded values, missing error handling, IAM over-permission, absent tests, and anything that would fail the adversarial checks listed in the Mentor Engagement Model
- Disagreement is resolved between author and reviewer first. If unresolved, it goes to the Wednesday session as a group discussion
- **Self-approval is not permitted.** A PR with only the author's approval is treated as a direct commit to `master`

Branch Protection Configuration

Configured on the repository in Milestone 1 and verified at the Milestone 2 checkpoint:

- Require a pull request before merging
- Require **at least 1 approving review**
- Require status checks to pass before merging — the CI pipeline is a required check

- Require branches to be up to date with `master` before merging
- Dismiss stale approvals when new commits are pushed
- Block force pushes to `master`
- Block deletion of `master`
- Protection rules apply to administrators too — no bypass

Why This Is Enforced

In a real team nobody pushes to the default branch. The habits being built here — small branches, descriptive commits, a PR that explains itself, a reviewer who actually reads it — are the difference between an engineer who can be trusted with a shared codebase and one who cannot. Reviewing a peer's booking engine will also teach more about concurrency than writing your own did.

Milestones and Review Checkpoints

Seven milestones across the project. Each has a fixed completion date by which the work must be done and committed. The **review conversation for each milestone is scheduled separately**, after the completion date, at a time that suits the mentor — it does not have to happen on the completion date itself.

Milestone 7 runs through to **Sunday 29 November 2026**, which is the final submission deadline. All work committed and merged by end of that day. Mentor reviews begin Monday 30 November.

#	Milestone	Working Days	Completion Date
1	Design and Repository Kickoff	Mon 12 – Fri 16 Oct	**Fri 16 Oct 2026**
2	Foundation, CI/CD and Authentication	Mon 19 – Fri 23 Oct	**Fri 23 Oct 2026**
3	Doctor Domain — Profiles, Availability, Slots	Mon 26 – Fri 30 Oct	**Fri 30 Oct 2026**
4	Patient Domain — Profiles, Search, Discovery	Mon 2 – Fri 6 Nov	**Fri 6 Nov 2026**
5	Booking Engine, Concurrency, Cancellation and Refunds	Mon 9 – Fri 13 Nov	**Fri 13 Nov 2026**
6	Reschedule Saga, Waitlist and Notifications	Mon 16 – Fri 20 Nov	**Fri 20 Nov 2026**
7	Observability, Web App, Hardening and **Final Submission**	Mon 23 – Sun 29 Nov	**Sun 29 Nov 2026**
—	*Mentor review period begins*	From Mon 30 Nov	—

Milestone 1 — Design and Repository Kickoff

Mon 12 – Fri 16 October · Completion date 16 October 2026

Goal: The complete design is on paper and defensible before any application code exists. No AWS resources created this week except an empty repository.

Work breakdown

- Mon–Tue: Domain modelling. Identify every entity, relationship, and access pattern
- Wed: Storage decisions. Decide what lives in RDS, what lives in DynamoDB, and why
- Thu: Concurrency and event design
- Fri: Diagram finalisation, repository setup, checkpoint

Deliverables

- Architecture diagram: all services, data flows, VPC layout, event topology, CI/CD pipeline
- RDS schema: every table with columns, types, indexes, foreign keys
- DynamoDB design: every table with partition key, sort key, GSIs, and the access patterns each serves
- **Concurrency strategy written down** — which record is the contended resource, what the conditional write looks like, how conflicts are retried, how retries are bounded
- **Timezone strategy written down** — how patterns are stored, how slots are materialised, how DST is handled
- **Reschedule saga designed** — step sequence, failure points, compensating action at each
- Event catalogue: every event name, payload shape, and consumer list
- API surface listed: every endpoint with method, path, role requirement
- Authentication design — token lifetimes, claim structure, refresh and revocation strategy, where the signing secret lives
- CI/CD pipeline design and GitHub secrets strategy
- Repository initialised with folder structure, README skeleton, and `.gitignore`
- **Branch protection configured on** `master`: PR required, one approving review required, status checks required, force push and deletion blocked

Services in focus: none deployed — design only

Checkpoint: Mentor reviews the design end to end. The participant must defend the concurrency strategy and the storage split specifically. **No implementation proceeds without sign-off on the concurrency approach.**

Milestone 2 — Foundation, CI/CD and Authentication

Mon 19 – Fri 23 October · Completion date 23 October 2026

Goal: Infrastructure deployed by the pipeline, not by hand. Both user types can register and log in. Everything from this point flows through CI/CD.

Work breakdown

- Mon: Network stack — VPC, subnets, route tables, security groups
- Tue: Data stack — RDS, DynamoDB tables, S3 buckets. GitHub Actions pipeline
- Wed: Custom auth Lambdas — register, login, refresh, logout
- Thu: Lambda Authorizer, role enforcement, test harness in CI
- Fri: Verification and checkpoint

Deliverables

- Network stack deployed via GitHub Actions: VPC, public and private subnets, IGW, route tables, security groups
- Data stack deployed via GitHub Actions: RDS MySQL in private subnet, all DynamoDB tables, S3 buckets
- GitHub Actions pipeline functional with Change Set-based deployment
- Registration endpoints for both roles, with bcrypt hashing at work factor 12 or higher and password policy enforced
- Login endpoints returning a signed JWT access token and an opaque refresh token
- Refresh and logout endpoints, with refresh tokens stored server-side in DynamoDB and revocable
- JWT signing secret provisioned as an SSM SecureString and read at cold start

- Lambda Authorizer verifying signature, expiry, and role, resolving `userId` and `role` into authorizer context
- Verified: tampered token returns `401`, expired token returns `401`, valid token for the wrong role returns `403`, token used after logout returns `401`
- Role enforcement proven — a patient token is rejected on a doctor endpoint
- IAM roles created for every planned Lambda function
- SSM parameters created for credentials and all configurable rules
- Verified: a test Lambda connects to RDS from inside the VPC
- Unit test harness wired into CI, failing the build on error
- CI pipeline registered as a **required status check** on `master`
- All Milestone 2 work delivered through feature branches and approved pull requests — no direct commits
- Secret scanning and IaC scanning active in the pipeline
- Structured JSON logging with correlation IDs

Services in focus: VPC, IAM, CloudFormation, API Gateway, Lambda, RDS, DynamoDB, S3, SSM Parameter Store, GitHub Actions

Checkpoint: Mentor reviews the GitHub Actions run log rather than the console. **Verifies branch protection settings and confirms every commit on `master` came through an approved pull request.** Registers a doctor and a patient live. Attempts a doctor operation with a patient token, a tampered token, and a post-logout token, confirming correct rejection in each case. Inspects the user table to confirm passwords are bcrypt-hashed. Spot-checks IAM policies and security group rules.

Milestone 3 — Doctor Domain: Profiles, Availability and Slots

Mon 26 – Fri 30 October · Completion date 30 October 2026

Goal: Doctors can fully manage their professional presence, and bookable slots exist — correctly, including across a DST boundary.

Work breakdown

- Mon: Doctor profile CRUD, photo upload via pre-signed URL
- Tue: Consultation type management, availability pattern endpoints
- Wed: Blackout dates, slot materialiser Lambda
- Thu: DST handling, idempotent materialisation, doctor dashboard first version
- Fri: Verification and checkpoint

Deliverables

- Doctor profile endpoints: create, read, update, with all required fields
- Profile photo upload via S3 pre-signed URL
- Consultation type CRUD with duration, fee, and capacity
- Weekly availability pattern endpoints
- Blackout date endpoints
- Slot materialiser running on EventBridge Scheduler, populating the rolling 30-day window into DynamoDB
- Materialiser verified **idempotent** — re-running a date produces no duplicates
- **DST correctness demonstrated** with a worked test case crossing a transition
- Doctor schedule day-view endpoint

- Doctor dashboard first version on EC2: profile editor, availability editor, schedule view
- X-Ray tracing enabled across the doctor API path

Services in focus: API Gateway, Lambda, RDS, DynamoDB, S3, EventBridge Scheduler, EC2, X-Ray

Checkpoint: Mentor reviews the DST test case line by line. Runs the materialiser twice on the same date and confirms no duplicate slots. Logs into the doctor dashboard and edits a profile and an availability pattern live.

Milestone 4 — Patient Domain: Profiles, Search and Discovery

Mon 2 – Fri 6 November · Completion date 6 November 2026

Goal: Patients can register, manage their profile, find doctors, and see available slots. Everything is ready for booking except booking itself.

Work breakdown

- Mon: Patient profile CRUD
- Tue: Doctor search with multi-attribute filtering and pagination
- Wed: Public doctor profile endpoint, slot availability lookup
- Thu: Patient dashboard first version — search, slot browser
- Fri: Verification and checkpoint

Deliverables

- Patient profile endpoints: create, read, update, delete
- Doctor search endpoint supporting filters on name, hospital, specialisation, and city, with pagination
- Search backed by appropriate RDS indexes — index choices documented and justified
- Public doctor profile endpoint returning consultation types and fees
- Slot availability endpoint returning free seats per slot for a date range
- Patient data isolation proven — a patient cannot read another patient's profile
- Patient dashboard first version on EC2: profile editor, doctor search with filters, slot browser
- Integration tests running against deployed infrastructure in CI

Services in focus: API Gateway, Lambda, RDS with indexes, DynamoDB, EC2

Checkpoint: Mentor searches for doctors using every filter combination. Attempts to read another patient's profile and confirms rejection. Reviews the chosen indexes against the actual query patterns. Logs into the patient dashboard and browses slots.

Milestone 5 — Booking Engine, Concurrency, Cancellation and Refunds

Mon 9 – Fri 13 November · Completion date 13 November 2026

Goal: Bookings are correct under contention and refunds are financially correct. **This milestone is the project.**

Work breakdown

- Mon: Booking endpoint, validation, capacity arithmetic
- Tue: Optimistic concurrency control, conditional writes, bounded retry
- Wed: Idempotency store, appointment persistence to RDS, event publication
- Thu: Cancellation, refund tier calculation, refund ledger

- Fri: Mandatory tests passing, verification and checkpoint

Deliverables

- Booking endpoint with full validation: slot exists, not in the past, patient limits respected
- **Optimistic concurrency control** on the slot record using conditional writes and a version attribute
- Bounded retry with backoff on conflict; retries counted as a metric
- Capacity enforced exactly, including multi-capacity Health Camp Sessions
- `409 Conflict` returned when the slot is full
- Idempotency store returning the original appointment on key replay
- Appointment persisted to RDS as system of record
- `AppointmentBooked` published to EventBridge
- Appointment listing and detail endpoints, scoped to the caller
- Cancellation endpoint with refund tier calculation from SSM-configured windows
- Doctor-initiated slot cancellation refunding all affected patients in full
- Refund ledger in RDS, exposed via a patient endpoint
- `AppointmentCancelled` and `SlotSeatFreed` published
- **Mandatory Test 1 — Concurrency:** 20+ simultaneous bookings at a capacity-3 slot, exactly 3 succeed, passing in CI
- **Mandatory Test 2 — Idempotency:** same key replayed, exactly one appointment, passing in CI
- Unit tests covering every refund tier boundary
- `BookingConflictRetries` and `CancellationsByTier` metrics publishing

Services in focus: DynamoDB conditional writes, Lambda, API Gateway, RDS, EventBridge, SSM, CloudWatch metrics

Checkpoint: Mentor runs the concurrency test live, then independently fires parallel bookings with their own script. Reviews the conflict-retry metric to confirm contention is genuinely occurring and being handled — **not avoided by accidental serialisation**. Tests every refund tier boundary including the exact cutoff instants. Reads the Concurrency Design Note.

Milestone 6 — Reschedule Saga, Waitlist and Notifications

Mon 16 – Fri 20 November · Completion date 20 November 2026

Goal: The distributed transaction is correct including when it fails midway, waitlist exclusivity is guaranteed, and every appointment change produces a notification.

Work breakdown

- Mon: Reschedule saga in Step Functions with compensation paths
- Tue: Waitlist join and leave, hold offer issuance on seat free
- Wed: Hold offer exclusivity, acceptance flow, expiry sweeper
- Thu: Notification system — EventBridge consumers, SQS buffering, SNS delivery, reminders
- Fri: Mandatory tests passing, verification and checkpoint

Deliverables

- Reschedule endpoint orchestrated by Step Functions

- Saga implemented: claim target seat, release original seat, with a compensating action if release fails after a successful claim
- Reschedule attempt limit and 6-hour cutoff enforced
- `AppointmentRescheduled` published on success
- Waitlist join, leave, and listing endpoints with the 5-waitlist cap enforced
- Hold offer issued to the first waitlisted patient on `SlotSeatFreed`
- **Held seats invisible to normal booking** for the hold duration
- Offer acceptance endpoint converting the hold into a confirmed appointment
- Expiry sweeper passing the seat to the next patient in line
- Notification consumers: EventBridge → SQS → Lambda → SNS, with a DLQ per consumer
- All eight notification events from the Frozen Rule Set delivering
- **Exactly-once dispatch** enforced through a DynamoDB dispatch log
- Reminder dispatcher sending at 24 hours and 2 hours before appointment
- No-show sweeper marking stale confirmed appointments
- **Mandatory Test 3 — Reschedule saga:** forced failure during release, patient holds exactly one appointment, passing in CI
- **Mandatory Test 4 — Waitlist exclusivity:** one freed seat, three waitlisted patients, only one live offer at any moment, passing in CI
- `ReschedulesCompensated`, `WaitlistPromotions`, `OffersExpired`, `NotificationsDispatched` metrics publishing

Services in focus: Step Functions, EventBridge, SQS with DLQs, SNS, DynamoDB with TTL, EventBridge Scheduler, Lambda

Checkpoint: Mentor forces a reschedule failure and verifies the patient holds exactly one appointment, then reviews the Step Functions execution history including the compensation branch. Frees a seat with three patients waitlisted and confirms only one offer is live and the seat is unbookable while held. Triggers the reminder dispatcher twice and confirms no duplicate notification.

Milestone 7 — Observability, Web App, Hardening and Final Submission

Mon 23 – Sun 29 November · Final submission Sunday 29 November 2026

Goal: The system is observable, the web app is complete, the platform is production-ready, and everything is committed and submitted by end of day 29 November.

Work breakdown

- Mon: CloudWatch dashboard, all custom metrics, alarms
- Tue: Doctor portal completed — all views and actions
- Wed: Patient portal completed — booking flow, waitlists, refunds. **Final Wednesday in-person session with mentors**
- Thu: IAM audit, drift detection, DR drill, cost review
- Fri: Teardown and full redeploy from scratch, documentation finalised
- Sat–Sun: Final merges to `master`, submission note, demo preparation

Deliverables

- `medislot-operations` CloudWatch dashboard populated across every required widget

- All custom business metrics publishing and visible
- All six alarm categories deployed and delivering to SNS
- X-Ray traces complete end to end for a booking and for a reschedule including compensation
- **Doctor portal complete:** profile, consultation types, availability, blackouts, day and week schedule, patient list per slot, outcome marking, slot cancellation
- **Patient portal complete:** profile, doctor search, slot browser, booking flow, upcoming and past appointments, active waitlists with pending offers, refund history
- Landing page role selection working, with route guards enforced in both directions
- IAM audit complete — no wildcard actions anywhere, every role scoped to specific resource ARNs
- All CloudFormation stacks reporting `IN_SYNC` under drift detection
- RDS automated backups and DynamoDB point-in-time recovery enabled
- **DR restore drill** performed and documented with a measured restore time
- AWS Budgets configured with cost allocation tags on every resource
- Cost review written
- **Full teardown and redeploy from scratch via the pipeline**, no manual steps
- All eleven deliverables finalised

Final submission — by end of day Sunday 29 November 2026

- Every feature branch merged into `master` through an approved pull request. No unmerged work
- No open pull requests remaining
- GitHub Actions pipeline **green on** `master`
- All documentation committed: architecture diagram, data model, design notes, API documentation, runbook, test report, DR drill record, cost review
- A submission note in the README confirming the system is deployed, with the web app URL and API base URL
- The repository is considered **frozen** at end of day 29 November. Commits after this point are not considered part of the submission
- 30-minute walkthrough demo prepared and rehearsed

Services in focus: CloudWatch, X-Ray, EC2, IAM, CloudFormation drift detection, AWS Budgets, RDS and DynamoDB backup

Checkpoint: No separate checkpoint for this milestone. Mentors review after submission closes on 29 November.

Mentor Review Period

Begins Monday 30 November 2026

Mentor reviews begin once the repository is frozen. Apprentices are not expected to make changes during this period, but should be available for their demo slot and for follow-up questions.

What mentors will do

- Review the repository: templates, Lambda source, pipeline configuration, test suites
- Review the commit and pull request history, including whether the branching policy was followed
- Attempt a fresh deployment following only the documented runbook, with no apprentice assistance
- Exercise every API endpoint, authenticated and unauthenticated

- Attempt to overbook a slot with their own concurrency script
- Attempt to act as another user by manipulating request bodies and tokens
- Test every refund tier boundary
- Run scheduled jobs twice
- Inspect IAM for wildcards, check RDS network exposure, scan git history for credentials
- Review the CloudWatch dashboard, alarms, and X-Ray traces

The demo must include

1. A patient registering, selecting "Login as Patient" from the landing page, searching for a doctor, and booking an appointment through the web app
2. The same appointment appearing in the doctor portal after logging in as the doctor
3. A token being rejected — expired, tampered, wrong role, and post-logout
4. The concurrency test run live, proving no overbooking
5. A cancellation showing the correct refund tier applied
6. A forced reschedule compensation
7. Waitlist exclusivity with multiple patients
8. A feature branch, its pull request with peer approval, and the pipeline run that deployed it
9. The CloudWatch dashboard with live data and one alarm firing
10. The architecture diagram walked against the running system
11. Two design decisions explained, and one the apprentice would reverse with hindsight

---

Mentor Engagement Model

Cadence

Engagement	Frequency	Duration
Milestone checkpoint review	After each milestone completion date, scheduled at the mentor's convenience	45 minutes
Group session with mentors, in person	**Every Wednesday**	60 minutes
On-demand doubt-clearing call	**Daily slot, 3:00 – 4:00 PM**	15 minutes, booked a day in advance
Concurrency deep dive	Once, during Milestone 5	60 minutes
Async support	Any working day	Same-day response

Weekly In-Person Session — Wednesdays

Mentors meet **all apprentices together, in person, every Wednesday**. This is the main synchronous touchpoint of the week.

Agenda:

- Each apprentice gives a brief update: what is done, what is in progress, what is at risk for the current milestone

- Open doubt clarification — anything technical, architectural, or process-related
- A short walkthrough of one apprentice's work in rotation, so everyone sees how others are approaching the same problem
- Mentors flag common mistakes observed across the group that week

Attendance is expected. If an apprentice cannot attend, they notify the mentor in advance and post their update asynchronously the same day.

On-Demand Doubt-Clearing Calls

Apprentices may book a **15-minute call with a mentor on any working day between 3:00 PM and 4:00 PM**.

- The slot must be **requested at least one day in advance**, so the mentor can look at the relevant code or stack beforehand
- The booking request must state the topic. "I have a doubt" is not sufficient — "I need help reasoning about why my conditional write is not failing under parallel load" is
- One slot per apprentice per day. If multiple apprentices book the same day, slots are split within the hour
- These calls are for **reasoning and unblocking**, not for live debugging sessions or code walkthroughs. Anything needing longer is scheduled separately

Async Support

Available on any working day through the group channel, with a response expected the same working day.

Async is the **first** channel, not the last. An apprentice should post a question asynchronously before booking a call — many questions resolve in a single exchange without needing synchronous time, and posting publicly means the whole group benefits from the answer.

Use async for: a specific error, a conceptual clarification, confirming an approach before committing to it, or a question another apprentice may have already hit.

Use a booked call for: something needing back-and-forth, a design decision with several viable paths, or anything where a written exchange has already gone two rounds without converging.

Escalation Path

1. **Try it yourself for up to two hours** — read the logs, read the trace, read the stack events
2. **Post asynchronously** in the group channel using the four-line format below
3. **Book the 3:00 PM slot** for the following day if the async exchange has not resolved it
4. **Raise it at the Wednesday session** if it is conceptual or likely shared by others
5. **Request a pairing session** if still blocked after the above — mentors will pair for 30 minutes

The Milestone 5 deep dive is scheduled rather than optional. The booking engine is where most apprentices will struggle, and a scheduled session removes the need for anyone to admit they are stuck before asking for time.

How to Ask for Help

Questions must be framed as four lines:

1. What I am trying to do
2. What I tried
3. The exact error message or observed behaviour
4. My current hypothesis about the cause

A question without a hypothesis will be returned with a request for one. The hypothesis is the learning.

Staying silently blocked for a full day is treated as a process failure, not a technical one. The channels exist to be used.

What the Mentor Does Not Do

- Write code, CloudFormation templates, or pipeline configuration
- Make architecture decisions — these can be challenged and stress-tested, never answered
- Debug Lambda or stack failures directly. Participants use CloudWatch Logs, X-Ray, and CloudFormation Events first
- Supply the concurrency control approach, the saga design, or the timezone strategy. These must be reasoned to independently
- Review code that has not been pushed to GitHub

What the Mentor Will Actively Try to Break

Participants should expect the mentor to behave adversarially at checkpoints. Specifically:

- Firing parallel requests at any contended endpoint
- Manipulating request bodies to act as a different user
- Replaying requests to test idempotency
- Testing exact boundary instants on refund tiers
- Running scheduled jobs twice
- Looking for hardcoded values, wildcards in IAM, and credentials in git history

This is stated up front so it is treated as a design constraint rather than a surprise.

Evaluation Criteria and Success Measures

Success Measures

A strong submission satisfies all of the following:

- The system deploys into a fresh account by running the pipeline, with no manual steps beyond configuring repository secrets and seeding the JWT signing secret
- **The reviewer cannot overbook a slot** under any concurrent load they apply
- Replaying an `Idempotency-Key` never produces a second appointment
- A forced failure mid-reschedule leaves the patient holding exactly one appointment
- A freed seat is never held by two patients simultaneously, and is unbookable while held
- Every refund tier calculates correctly, including at the exact boundary instants
- A doctor-initiated cancellation always refunds in full regardless of timing
- Slot materialisation is correct across a DST transition, demonstrated with a test case
- Re-running any scheduled job produces no duplicate effects
- Reminders fire exactly once per interval per appointment
- Every endpoint rejects unauthenticated requests. Patients cannot act as other patients. Doctor endpoints reject patient tokens
- Both dashboards are functional and a complete booking journey can be performed through the patient UI

- The CloudWatch dashboard is populated with live data across all widgets
- At least one alarm fires and delivers during evaluation
- No IAM role contains a wildcard action
- All CloudFormation stacks report `IN_SYNC`
- The DR drill is documented with a measured restore time
- Commit history is distributed across all seven weeks
- Every change on `master` arrived through a pull request with at least one peer approval and a green pipeline
- `git log master --first-parent` shows merge commits from feature branches, with no direct commits
- An expired, tampered, wrong-role, or post-logout token is rejected with the correct status code
- Passwords are stored as bcrypt hashes at work factor 12 or higher

Distinction Indicators

- OIDC federation rather than long-lived access keys in GitHub secrets
- A second environment deployed from the same templates with different parameters only
- API versioning with a breaking change shipped while the previous version remains functional
- A live schema migration with backfill, applied through the pipeline
- DLQ depth alarms triggering an automated handler that logs failed message detail and publishes a metric
- Search performance measured and index choices revised based on evidence

What Will Cause Failure Regardless of Score

- The reviewer successfully overbooks a slot
- A forced reschedule failure leaves a patient with zero or two appointments
- Any AWS credential committed to the repository at any point, including deleted commits and abandoned branches
- RDS reachable from the public internet
- Any resource created or modified by hand after Milestone 2
- Authentication missing or bypassable on any endpoint
- A patient able to read or modify another patient's data, or reach a doctor-only operation
- Concurrency control implemented by serialising all bookings through a single-threaded bottleneck. The requirement is correctness **under concurrency**, not the avoidance of concurrency
- Deployment requiring steps not documented in the runbook
- Commit history consisting of a single bulk commit, or commits appearing only on milestone completion dates
- **Direct commits to `master`**, or branch protection disabled or bypassed at any point
- Pull requests merged without a peer approval, or self-approved
- Storing passwords in plaintext, reversibly encrypted, or as an unsalted hash
- The JWT signing secret present anywhere in source control

> **A note on cost:** All services in this project operate within AWS Free Tier when used as specified. RDS `db.t3.micro` single-AZ and EC2 `t2.micro` are free for twelve months on new accounts. DynamoDB, Lambda, API Gateway, SQS, SNS, EventBridge, Step Functions Standard, X-Ray, and S3 all carry free tiers sufficient for this workload. Because authentication is hand-built rather than using a managed identity service, there is no additional identity cost at all.
>
> **Do not create customer-managed KMS keys** — use AWS-managed keys (`aws/ssm`, `aws/rds`) throughout. A single customer-managed key costs one dollar per month and is unnecessary here.
>
> The rolling 30-day slot window generates a meaningful row count. Keep the number of doctors and availability patterns small during development to stay inside the DynamoDB free tier. AWS Budgets is a required deliverable precisely so spend stays visible. The deployment runbook must include a full teardown section, and participants are responsible for deleting all resources once evaluation is complete.